牛链 NIU 白皮书



V2.0 · 点对点工作量证明与可编程账户系统

2026 年 10 月 15 日 · 技术设计与实现状态版

摘要

牛链提出一个供个人与朋友学习、交流的独立区块链系统。参与者通过完整节点共同验证账本，矿工以 Ethash 工作量证明竞争出块，NIU 是支付转账与合约执行费用的原生资产。系统保留 EVM 执行能力，以 Windows 桌面钱包提供账户管理、收款二维码、发送地址簿、挖矿控制和加密备份。

本文参考中本聪《Bitcoin: A Peer-to-Peer Electronic Cash System》讨论问题与机制的方式，但不是比特币白皮书的替换名称版本。牛链使用账户与 nonce 模型，并加入 Gas、合约状态和本机钱包安全边界，不能直接照搬比特币的 UTXO、coinbase 成熟期或简化支付验证结论。[1]

本文描述的基础发行规则已进入修改后的节点源码，并在本机独立节点中验证；公共网络尚未启动，两台实体电脑互联尚未验收。安全扫描发现了必须处理的旧上游依赖问题，因此本版不是“安全审计通过”或“主网正式发行”的声明。

1. 问题、目标与安全前提

两位朋友各自维护一份余额表，不能仅凭对方发来的截图判断付款是否有效。系统需要解决：交易是否由资产控制者授权、同一余额是否被重复支出、离线节点如何恢复一致历史，以及谁有权决定新的发行。牛链使用数字签名验证授权，以确定的状态转换规则检查余额与 nonce，以工作量证明选择有效历史。

项目目标是独立运行一条可挖矿、可转账、可编程的链，并提供普通 Windows 用户能够操作的钱包。它不是在现有链上部署一个同名代币，也不是把 GitHub 当作区块链服务器。朋友之间无需向项目方提交私钥，项目方也不应提供找回私钥或替用户撤销转账的承诺。

安全依赖若干前提：随机数来源可靠，签名实现正确，用户终端没有被恶意程序控制，多数有效算力不合谋攻击，参与者执行相同规则，并能接触到足够独立的网络信息。这些前提不能由 Logo、代码数量或节点数量单独证明。任何前提失效都必须在威胁模型中单独处理。

牛链不承诺价格、收益、交易所上线或资产兑换。技术可运行、软件已上传、网络已启动和资产存在市场价值是不同事实。本文给出的目标与计算不构成这些事实的替代证据。

2. 交易与账户所有权

牛链采用 EVM 账户模型。普通账户由私钥控制，公开地址用于收款。账户状态包含余额和交易序号 nonce；合约账户还包含代码与存储。一次交易需要明确网络、发送者序号、接收地址、金额、执行资源限制、费用参数和可选合约数据。

节点接收已签名交易后，应验证编码、签名、链标识、nonce、余额和费用约束。签名证明对特定交易内容的授权；它不证明发送者的真实身份，也不证明交易一定会被矿工收录。普通用户获得地址或二维码，并不因此获得对应资产的支出权限。

同一发送账户的两笔交易如果使用相同 nonce，不能都按原顺序在同一条有效历史中执行。交易池替换规则、区块顺序及链重组会影响哪笔交易最终入块。钱包必须区分“已广播”“已入块”“执行失败”和“重组后待确认”，不能把点击发送当成付款完成。

交易签名沿用 ECDSA/secp256k1。签名包含链上下文以降低跨链重放风险，但链 ID 不是所有误转账的万能防线：普通地址外观与其他 EVM 链相同，用户仍可能向错误网络或错误地址付款。钱包必须显示网络与完整接收地址，签名前再次确认。[3]

当前普通转账入口限制为普通账户，不把向合约地址付款包装成普通转账；合约通过单独入口交互。这是产品范围限制，不代表 EVM 协议不能进行其他形式的合约调用。

3. 区块、状态承诺与历史顺序

每个区块通过父区块哈希引用上一块，并承诺交易、收据和执行后的状态。修改已确认历史会改变相应承诺，攻击者需要构造一条满足全部验证规则、并具有足够累计工作量的竞争分支。节点不能因为对方提供了一份余额 JSON 就更新自己的资产状态。

比特币交易的 Merkle 承诺与牛链的状态承诺承担相关但不同的职责。牛链继承所选 EVM 客户端的交易、收据和状态树结构，不把比特币的 80 字节区块头、UTXO 裁剪方法或存储增长估算直接套用到本项目。合约存储、日志和状态维护都会产生额外成本。

完整节点逐步执行收到的区块，对交易合法性、状态转换、区块奖励和工作量证明进行检查。有效链选择比较累计有效工作量，不是按节点投票数量，也不是简单选择区块数量最多的一支。无效签名或越界增发不能仅凭优势算力就变成符合既定规则的交易。

应用层仍需面对重组。原来已入块的交易可能离开主链，也可能随后重新入块。钱包本轮增加了收据复核与确认数展示；这不等于已完成所有深度重组、数据库恢复和跨节点异常场景的测试。

4. 工作量证明与难度调整

矿工构造候选区块，改变工作量证明输入，寻找满足目标难度的有效结果。其他节点验证工作量证明和区块内容。计算工作不应使无效交易、错误状态根或超额奖励被接受。牛链首版候选采用 Ethash，不把现今已经改用 PoS 的 Ethereum 主网描述为仍在挖矿。[2]

Ethash 需要缓存与数据集。CPU 可以用于本项目功能验证，但算法并非专为普通 CPU 公平竞争而设计，也不能保证抵抗专用矿机。相同线程数不代表相同算力，短期出块存在随机性。RandomX 更偏向通用 CPU，但接入当前客户端需要另一轮共识、矿工、跨平台一致性和性能验证，未作为本版的既定实现。

目标平均出块时间约为 60 秒。当前难度候选沿用 Ethash/EIP-100 的逐块调整形状，去掉叔块项与难度炸弹。设父难度为 D，父子时间差为 Δt，则实现采用以下整数关系：

```
a = max(1 − floor(Δt / 42), −99)
D_next = max(131072, D + floor(D / 2048) × a)
```

42 秒是时间桶宽度，不是承诺的出块间隔。在理想化指数到达模型中，此调整形状的平衡均值接近 42/ln(2)，约 60.6 秒；真实网络受到难度下限、算力变化、时间戳、传播延迟和整数截断影响。这只是候选参数解释，不能替代长期模拟或安全证明。

尤其在新链低算力或难度下限附近，平均速度可能明显偏离目标。正式启动前必须模拟算力突然退出、优势算力加入、长时间停机、异常时间戳和追赶过程。当前节点继承上游时间检查，新难度策略仍需审查。本文不宣称该策略已经具备比特币或成熟网络同等的攻击分析深度。

5. 节点发现、同步与点对点网络

安装程序本身不形成在线节点。钱包启动内部完整节点后，才参与连接、验证、保存与传播。两台电脑必须使用相同创世块、链 ID、网络 ID 和共识规则，并具备可达的通信路径。开启挖矿是另一个主动操作；没有矿工继续出块时，交易不能获得新的区块确认。

首轮采用静态引导节点配置。新节点从预置或用户核实的 enode 找到在线节点，客户端维护连接，保存节点身份与本地链数据库，后续启动补齐缺失历史。当前关闭开放式节点发现，不宣称已经具备狗狗币同等规模的 DNS 种子、地址传播和独立节点生态。[4]

引导节点帮助接入，不拥有修改别人余额或签名交易的权限。它仍可能造成单点故障或向新用户提供偏置的网络视图，因此后续需要不同运营者、不同网络的多个入口。客户端不能仅因为连上一个入口，就宣称全网同步完成或去中心化安全已经成立。

P2P 端口用于节点通信，钱包控制、签名和节点管理不应随之对公网开放。本轮已把管理与挖矿控制移到本机 IPC；HTTP RPC 仅保留查询和公开交易相关命名空间，并绑定回环地址。对外监听、路由器映射和防火墙变更必须由明确配置管理，不自动打开服务器权限。

节点可以离线再加入。已有数据库保留有效进度，重新连接后下载缺失区块、验证并重新计算状态。钱包区分未联网、存在同步积压和已连接。没有同步积压不等于已经知道全网最新高度；在隔离网络中，节点仍可能不知道另一条分支。

GitHub 只分发源码、资料和安装包。它不承担区块传播和持续出块，也不会因为仓库存在就让两个互不相连的钱包自动共享余额。两台都在 NAT 后面的电脑，可能仍需要一个可达入口才能方便互联。

6. 发行、奖励与手续费

以下规则是小袁已确认的实现方向。对集成链 ID 26261017，修改后的节点源码执行零创世分配、无创始人预留、初始 1000 NIU 区块补贴、每 105 万块减半、18 位基础单位，以及禁止叔块和叔块奖励。正式网络与发布时间仍未冻结。

参数	规则
原生资产	牛币 NIU；简称不构成同名项目或商标的独占声明
上限目标	基础补贴总额不超过 21 亿 NIU
基础单位	1 NIU = 10 的 18 次方个整数单位
创世与预留	分配为 0；创始人预留为 0
第一阶段奖励	每个有效普通区块 1000 NIU
减半间隔	1050000 块，以高度计算，不以日历时间计算

参数	规则
额外增发	不允许叔块或叔块包含奖励；不设置随意增发入口
出块目标	平均约 60 秒；候选难度策略仍待完整验证

令 h 为区块高度，U 为每 NIU 的基础单位，H 为减半间隔：

```
U = 10^18; H = 1,050,000
R(0) = 0
R(h) = floor((1000 × U) / 2^floor((h-1)/H)), h ≥ 1
```

第 1 块至第 1050000 块每块获得 1000 NIU，第 1050001 块开始获得 500 NIU。全部计算采用整数单位，不使用浮点数。70 个非零阶段的基础补贴合计为 2099999999.99999999996955 NIU，低于 21 亿 0.00000000003045 NIU，不另行增发补齐舍入差额。

第一阶段发行 10.5 亿 NIU，第二阶段新增 5.25 亿 NIU。在理想一分钟速度下，每阶段约 1.998 年；真实时间取决于实际出块。零预挖不等于参与机会或结果必然公平，先上线、先获得算力或掌握更多硬件仍会形成优势，启动过程应公开且可复核。

交易费用与新增发行分开。London EVM 路径下基础手续费销毁，矿工小费是既有余额的转移。总补贴累计、销毁后余额总和、单个钱包余额不是同一指标。智能合约可以发行自己的应用代币，但不能因此获得任意增发原生 NIU 的权限。

当前账户模型的区块奖励计入状态余额，尚未实现比特币式 coinbase 成熟锁定。不能把原先讨论过的“奖励成熟后支出”描述为已完成共识功能。确认提示能说明重组风险，不能代替共识层的成熟限制。若要加入这种限制，必须先明确规则并修改、测试相应状态转换。

7. 存储、恢复与磁盘成本

节点保存账本和执行所需的数据，钱包保存控制资产的密钥与本地记录，两类数据不能互相代替。删除链数据库通常可以从其他节点重新同步；删除唯一密钥且没有有效备份，则不能依靠区块历史找回支出权限。

当前使用完整同步路径。合约状态、交易收据和历史会随使用增长，不承诺固定安装体积，也不承诺仅保存区块头就足以完整验证本项目所有合约执行。裁剪、快照和轻客户端可以成为后续工程，但需要单独定义验证与恢复边界。

节点数据库异常退出、磁盘写满和升级格式不兼容，可能导致服务中断。程序正常关闭时停止矿工、退出节点并保存状态。正式版还需覆盖故障注入和恢复演练。当前 Windows 上游缓存清理用例存在失败，不能用功能演示掩盖该问题。

如果全部节点的历史数据都丢失且没有副本，只有私钥备份不能重新生成所有历史交易。因此独立节点与合适的链数据备份也有意义，但链数据备份不能包含他人的私钥来冒充“同步包”。

8. 支付验证与确认含义

当前 Windows 钱包内置完整节点，查询本机已经验证的链状态。本文没有承诺已实现类似比特币 SPV 的独立轻钱包，也没有把某个服务器返回的余额视为同等验证。未来若提供轻客户端，需要明确证明格式、可信根、数据可用性和攻击模型。

普通用户看见一笔交易时，应区分交易池可见、区块执行成功和后续确认增长。交易收据中的执行失败可能仍消耗 Gas；没有收据并不总等于“从未广播”。网络延迟或等待超时之后直接重复发送，可能造成另一笔付款，正式钱包需要更完整的待确认交易管理。

本轮实现会复查本地记录的交易收据，遇到原已入块交易离开当前链时显示重组待确认。收据再次出现后更新区块与确认数。这只是钱包的状态呈现，深度分叉测试、交易替换和多节点视图冲突仍需继续验收。

不采用一个固定确认数作为所有规模网络的绝对安全承诺。少数朋友运行的低算力网络，面对外部优势算力时可能很脆弱，即使短时间积累了很多区块，也不能推导为高价值付款已经安全。

9. 多地址、金额与发送地址簿

一个钱包可以保存多个自有账户。当前上限为 32 个本地地址；新增收款地址创建新的随机密钥，经过加密落盘后才向界面返回。它不是换一个二维码图片，也不等于复制原地址的别名。地址备注与密钥所有权是不同信息。

接收页面可以选择自有地址、显示完整地址、生成对应二维码并复制地址。二维码当前编码公开地址，不包含私钥，也不携带金额或完整的跨链付款指令。扫描结果必须与界面地址一致，普通相机或其他链的钱包不因此自动兼容牛链。

发送地址簿保存朋友的公开地址和备注，最多 100 条。保存对方地址不意味着导入对方账户，也不会获得其余额控制权。用户选择联系人后仍应核对完整地址，尤其要防范剪贴板替换、相似首尾地址和设备上的恶意程序。

发送时选择自己的一个账户签名。概览可以汇总所有自有地址的余额，但账户余额不能像比特币 UTXO 输入那样自动合并为同一笔普通交易资金。所选账户必须独立承担付款与手续费。自动归集属于另一个需要费用与隐私分析的功能，本版未实现。

新增地址后必须重新导出备份。此前备份不可能包含之后才创建的随机私钥。当前不是基于助记词的确定性地址派生方案，不能承诺一份旧助记词就能恢复未来全部地址。软件通过“未备份地址”提示降低这种误解。

10. 私钥、加密备份与隐私

运行密钥目前使用 Windows DPAPI CurrentUser 保护，在钱包启动后由本机后台解密并用于签名。该机制约束于 Windows 用户上下文，不是独立的钱包口令锁。拥有相同用户权限的恶意程序、管理员入侵、屏幕控制或进程注入仍可能威胁资产。[5]

可携带 JSON 备份使用 ethers 的标准 V3 keystore 实现：scrypt 参数 N=131072、r=8、p=1，AES-128-CTR 加密，并验证对应 MAC。备份包含全部当前自有账户，保存后会解密回读核对地址。密码至少 12 个字符，但长度检查不等于强密码保证，仍应使用难以猜测的独立口令。[6]

恢复入口先检查格式、网络标签、账户数和 KDF 参数，再执行解密，避免恶意备份指定极端计算参数。恢复只允许进入全新数据目录，不覆盖已有钱包或链数据。密码不通过命令行参数、环境变量或日志传递。明文密钥在钱包运行期间保留在后台进程内存中，口令处理也会使用内存，不能承诺抵抗恶意终端或内存取证。

备份网络标签用于防止误操作，不是 V3 keystore MAC 对整个外层元数据的认证。当前备份不包含完整链历史、全部本地交易记录、联系人备注或合约应用配置；这些内容不能被描述为已经完整恢复。用户必须保留备份文件和密码，项目方不应代为托管私钥。

链上地址并非匿名身份保证。转账关系、合约行为、Gas 支付与重复使用地址可能产生关联；创建新地址也不能保证无法被关联。不要将身份证明、手机号、口令或其他秘密写入公开留言合约。

11. 威胁模型与攻击分析

PoW 的安全性涉及算力分布，而不是“钱包数量越多就一定安全”。分布在独立运营者和不同网络中的完整节点能够提高可用性与验证分散程度；同一人运行大量节点并不能等价增加诚实算力，也不能单独阻止优势算力重组。

参考比特币对追赶过程的讨论，在固定算力比例、独立出块等理想假设下，令 p 为诚实方出块概率、q 为攻击者概率。若攻击者已经落后 z 个区块，且 p>q，最终追平的简化概率为 (q/p)^z；当 q≥p 时，不能依赖这一衰减结论。这个“从已知落后状态开始”的模型，不等于收款人在等待期间遭遇双花的完整概率模型。[1]

牛链没有测得稳定的全网算力分布，不能把上述公式代入虚构参数给出“六次确认绝对安全”的结论。节点被隔离、网络分区、低算力突然退出和时间戳操纵，也会使理想模型与实际情况明显不同。

威胁	现有措施	仍需完成
未授权付款	本机签名、地址校验、转账确认	独立口令锁与自动锁定、完整费用预览
私钥与备份泄露	DPAPI、密码加密备份、错误口令拒绝	恶意终端防护、恢复体验与独立审查
跨站调用钱包	回环监听、会话令牌、Origin/Host 检查、请求限制	更全面的桌面与浏览器攻击面审查
节点控制暴露	admin/miner 使用本机 IPC，HTTP 拒绝对应方法	IPC 权限、最小权限部署与系统实测
链重组与双花	有效性验证、累计工作量、收据复核	深度重组、算力变化与隔离网络测试
依赖漏洞	固定依赖、npm 审计与 Go 二进制扫描	旧节点依赖修复及重新构建验证

截至本版检查，钱包 npm 依赖升级并覆盖修复后，注册表扫描报告 0 条已知漏洞；这只说明该扫描源未报告匹配项。Go 二进制扫描报告 41 条涉及候选程序所含符号的漏洞记录，涉及 6 个模块及标准库。二进制模式不提供完整网络可达性证明，仍足以阻止把候选程序作为正式安全版发布。详细记录随源码公开，不能以“扫描工具可能误报”为由直接忽略。

12. 智能合约与可编程能力

牛链保留 London EVM 执行路径，合约可使用 Solidity 编写并消耗 Gas。节点对相同输入执行相同规则，合约状态与交易结果纳入区块验证。链上程序不是普通操作系统，不会直接代替任意 Python 程序、网站后端或外部数据库。

目前已验证留言合约的部署、读取和更新，并在独立节点间同步。合约示例证明可编程路径，不证明所有应用已经安全，也不等于兼容所有最新 Ethereum 指令、钱包插件、桥接协议和开发框架。编译器目标与执行版本必须一致。

合约访问控制、重入、错误授权、不可控循环和外部价格来源都可能导致损失。原生链采用正确签名，并不自动修复应用合约漏洞。正式提供合约浏览或交互入口时，还需区分用户授权、调用数据、执行结果和费用，不把“消息签名”与“资产交易签名”混在一起。

普通 NIU 转账与智能合约拥有不同的风险边界。本版不提供跨链桥、去中心化交易所或收益合约的安全背书，也不因示例合约能够运行就承诺这些应用已经完成。

13. 密码学与抗量子演进

挖矿算法、交易签名与本地文件加密解决不同问题。Ethash 是工作量证明；secp256k1 用于当前交易授权；DPAPI 与备份加密用于密钥存储。把 Ethash 换成 RandomX，并不会自动使钱包签名具备抗量子能力。

本版不是抗量子链。未来可以评估 ECDSA 与 NIST ML-DSA-65 的混合签名，但需要定义交易编码、账户密钥绑定、签名域、验证成本、备份、轮换与迁移规则。若采用混合方案，应明确两种签名均需通过，不能静默降级成任意一种通过即可。[7]

EVM 执行兼容与外部钱包交易格式兼容是不同层面。新的签名格式可能需要定制客户端与 RPC；旧合约中的 ecrecover 或传统签名授权不会自动变成抗量子。标准化算法不代表本项目实现、钱包、网络和合约已经通过安全审计。

ML-KEM 是密钥封装机制，不应用作交易数字签名的替代名称。任何密码学升级都应使用成熟实现、官方测试向量和独立评审，不能自创算法后以“更先进”为理由跳过验证。

14. 钱包产品与二维纸币风格

Windows 钱包保留概览、发送、接收、交易记录、挖矿、智能合约、备份与设置七个主要页面。界面以普通操作对象组织，不要求用户自行输入 RPC 或运行开发命令。启动节点、开始挖矿和发起转账是三个有不同状态与错误反馈的动作。

视觉使用原创小犟牛币章：平面圆框、雕刻线条、米白底、深墨绿与少量古金色。借鉴纸币与常见动物币的视觉语言，不复制美元法定货币图样，不使用 Dogecoin 动物商标，也不暗示任何机构、国家或既有币种背书。币章是品牌表达，不属于安全机制。

钱包不自动挖矿，当前入口每次主动开启两个线程并限制单轮时长。正式版还需完善线程与资源设置、长时间运行体验、费用估算、锁定解锁和升级恢复。下载包不应包含开发者钱包、已有余额、访问令牌或服务器凭据。

15. 版本、开源与发布条件

集成验证网络使用链 ID 与网络 ID 26261017，创世哈希为 0x9590d27e9e5977eb00bf1fd794d93fce0cac22dad04dc3f25fc072ab0c87e053。|旧本机实验网络 26261015 沿用上游奖励，不得与按新规则生成的余额混用。集成网络不因本文列出参数而自动成为已上线主网。

源码基于 Core-Geth v1.12.23 起始提交 96b2afc25c7023bb500d14e8ed69455339b1ca90，并保留上游版权与许可。仓库提供对应修改源码、规则测试、钱包源码和构建说明，不将复制的上游代码计为新增开发量。程序签名、文件哈希和独立审计分别解决来源、完整性和审查问题，不能互相替代。

当前可提供源码与设计资料。正式安装包发布前至少需要：修复并复核节点依赖漏洞；完成两台实体电脑互联、同步、双向转账、离线恢复；验证难度与重组异常；完善独立钱包口令锁和恢复体验；检查费用、发送确认与余额边界；生成对应源码、签名与校验文件；核对首次启动不携带任何开发者密钥。

规则升级需要明确生效边界与兼容策略。改变发行、难度或交易规则可能形成不兼容分叉；不能通过普通界面更新静默改变已运行网络的资产规则。项目不能承诺“源码已公开就没有漏洞”，也不能把内部自检称为第三方审计。

16. 当前证据与后续验收

本轮修改后节点已经完成 Windows 编译。真实 PoW 集成验证使用同一台电脑上的两个独立进程、数据目录、节点身份和钱包，验证了零创世分配、每块 1000 NIU 基础奖励、相同区块哈希、双向转账、新地址签名支出、完整备份，以及依赖升级后的合约部署与更新。它不是两台实体电脑互联的替代证明。

模块验证覆盖错误口令、密文损坏、网络标签错误、异常 KDF 参数、中文路径与口令、恢复后签名一致、拒绝覆盖已有钱包，以及节点配置与创世同步状态边界。HTTP 边界验证覆盖错误 Host、跨站来源、缺少会话令牌、错误 Origin、表单内容类型、畸形与超长请求以及错误信息不回显秘密。

上述证据证明的是具体实现路径和有限边界。完整上游测试仍存在 Windows 缓存数量失败，Go 漏洞修复、跨电脑网络、深度重组和独立安全审查仍未闭环。代码行数、开发时间估算和漂亮界面不能改变这些完成边界。

参考资料

[1] Satoshi Nakamoto. Bitcoin: A Peer-to-Peer Electronic Cash System. https://bitcoin.org/bitcoin.pdf

[2] Ethereum 官方文档：Ethash。https://ethereum.org/en/developers/docs/consensus-mechanisms/pow/mining/mining-algorithms/ethash/

[3] Ethereum 官方文档：账户。https://ethereum.org/en/developers/docs/accounts/

[4] Dogecoin Core 官方源码：net.cpp、net_processing.cpp、chainparams.cpp、wallet/wallet.cpp。https://github.com/dogecoin/dogecoin

[5] Microsoft：DataProtectionScope。https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.dataprotectionscope

[6] ethers v6：Wallet 与 JSON Keystore。https://docs.ethers.org/v6/api/wallet/

[7] NIST FIPS 204：Module-Lattice-Based Digital Signature Standard。https://csrc.nist.gov/pubs/fips/204/final

[8] Geth 官方文档：JSON-RPC Server。https://geth.ethereum.org/docs/interacting-with-geth/rpc

[9] Go Vulnerability Database 与 govulncheck。https://vuln.go.dev/ · https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck

本文依据截至本版检查的代码、运行记录和资料编写。后续修复或设计变更应更新文档与证据，而不是将本版中的待完成事项自动解释为已经完成。